A Cup of Water: Interactive Multi-State Fluid Rendering

Project Proposal

Group 8
Ruiliang Gong, Runsheng Liu

HKUST - COMP5411 - Fall 2026

Background

Ruiliang Gong's graphics programming background comes primarily from COMP5411 and related programming coursework. In this project, he will focus on the GPU height-field simulation, interaction forces, and the coupling between bubbles, floating objects, and the water surface.

Runsheng Liu is also developing real-time graphics experience through COMP5411 and prior programming coursework. He will focus on scene construction, multi-state material rendering, user interaction, and the integration and performance testing of the complete system.

Project description

A Cup of Water: Interactive Multi-State Fluid Rendering is a real-time graphics project that turns an ordinary glass of water into a compact interactive simulation. The user can move the glass horizontally and observe the water continue to slosh because of inertia, touch the surface with the mouse to create local ripples, and drop objects into the glass. An impact produces waves, while an object's physical properties determine whether it floats or sinks. Together, these interactions establish a responsive liquid surface that the user can directly manipulate rather than merely watch as a precomputed animation.

The central idea is that one temperature parameter controls the behavior and appearance of the entire scene. Near room temperature, the system shows transparent liquid water with ordinary surface motion. As the temperature rises, bubbles form near the bottom, grow more active, rise, and disturb the surface when they arrive. Their motion also affects immersed or floating objects, so a previously stable object begins to bob as the water approaches boiling. During cooling, ice spreads gradually across the surface. The growing frozen region damps waves, restricts object motion, and eventually traps objects when the water reaches a solid state. Heating and cooling therefore change the simulation itself, rather than switching between unrelated visual effects.

Relationship to existing work and contribution. Height-field wave solvers, particle systems, simplified buoyancy models, and reflective or refractive water shaders are established real-time graphics techniques [1, 2, 3, 4, 5]. Three.js also provides water materials and interactive water examples, so implementing an isolated ripple effect or transparent surface would not by itself be new [6, 7]. We will build on these techniques, but implement and integrate our own surface update, interaction forces, bubble system, freezing mask, and multi-state shaders. Our main contribution is the shared temperature-driven coupling: bubbles generated by heating perturb the simulated surface and floating objects, while a spatially growing ice state changes local wave dynamics, object freedom, and material appearance. Existing components become different states of one responsive system rather than independent demonstrations. The core deliverables are this coupled behavior, object interaction, and multi-state rendering at an interactive frame rate. A full thermodynamic or volumetric fluid solver is outside our scope. Exterior condensation with interactive wiping remains a stretch goal.

Key challenges

- **Implementing an established surface method as a working baseline.** Height-field simulation is not our novel contribution, but it is a substantial hands-on foundation [1, 2]. We will implement the GPU state buffers, damped wave update, glass boundary conditions, normal reconstruction, and impulse injection ourselves. The same grid must represent broad sloshing caused by cup motion and smaller disturbances from mouse input, object impacts, and bubbles without becoming numerically unstable.
- **Designing the novel cross-system coupling.** The main technical contribution is the connection among temperature, bubbles or ice, the water surface, objects, and rendering. An object impact must excite the surface; surface motion must change a floating object's position; bubbles must perturb both; and freezing must progressively reduce motion. These subsystems operate at different spatial and temporal scales, so their forces and transition rules must be designed and tuned without unstable feedback or abrupt state changes.
- **Extending standard particle animation into coupled boiling.** Particle emitters already provide efficient bubble animation [3]. Our additional work is to make bubble count, size, and rise activity respond continuously to temperature, detect arrival at the moving surface, and convert each arrival into a localized surface impulse. The aggregate impulses must visibly affect the liquid and nearby objects without overwhelming the height-field solver.
- **Progressive freezing as both simulation and rendering state.** Many simple demos change only a material when water freezes. We instead need a spatial ice mask that grows over time, increases local damping, reduces surface mobility, and eventually constrains objects. The boundary between liquid and ice must remain stable and visually continuous during partially frozen states.
- **Adapting existing rendering models into a coherent transition.** Fresnel reflection, refraction, transparency, and physically based material parameters are established tools [5]. The challenge is to combine them without confusing depth artifacts, keep bubbles readable inside transparent water, and use the same simulated state to blend continuously from moving water to rougher and less transmissive ice.
- **Hands-on implementation and AI assistance.** The team will design the coupling rules, implement the shaders and simulation code, integrate the subsystems, tune parameters, profile performance, and construct test scenarios. We may use generative AI to locate documentation, explain unfamiliar API behavior, suggest small code fragments, and help diagnose errors. We will review and understand all generated material, adapt it to our design, and verify it through controlled tests. No AI model will generate the simulation at runtime or replace our implementation of the core algorithms.
- **Performance and scope control.** Surface updates, object physics, bubble particles, and transparent rendering all compete for each frame's time budget. We must choose grid resolution, bubble count, and shading complexity carefully, compare the coupled system with simpler uncoupled baselines, and preserve the core deliverables before attempting condensation.

Timeline

Week	Milestone	Details / Tasks
1	Proposal (Sep 29)	Finalize scope, architecture, ownership, and success criteria
2	Baseline and scene	Review existing methods; build the glass, controls, and base materials
3	Water surface	Implement the GPU height field, boundary handling, and normal updates
4	User interaction	Add cup-driven sloshing, mouse impulses, and damping controls
5	Project progress video	Demonstrate stable water; integrate object impacts, floating, and sinking
6	Heating	Add heat-driven bubbles, rising motion, and surface impulses
7	Cooling	Add ice growth, wave damping, object trapping, and material blend
8	Project presentation	Present coupled states and compare them with uncoupled baselines
9	Refinement	Tune coupling and shaders; profile performance; document AI assistance
10	Project final report	Fix defects, record the demo, and complete the report and presentation

Design and implementation details

The simulation will use a regular two-dimensional height field aligned with the water surface. Two GPU buffers will store the current and previous height values, and a shader will advance a damped wave equation each frame. This follows a well-established real-time water approach [1, 2], but we will write the update and interaction logic rather than treating an existing water demo as the finished project. Cup acceleration will add a low-frequency directional disturbance, while mouse contact, object impacts, and bubbles will inject localized impulses into the same field. Surface normals derived from neighboring height samples will drive lighting and refraction, ensuring that every interaction affects both motion and appearance.

Object behavior will use a lightweight rigid-body approximation suitable for the small scene. The submerged fraction of an object will determine an upward buoyancy force, while relative motion against the water will produce drag [4]. Contact with the surface will create an impulse whose strength depends on impact speed. The sampled surface height and velocity will then make floating objects respond to waves. This approach will not reproduce full fluid displacement, but it gives clear and controllable feedback at a practical cost.

Temperature will act as a shared state variable rather than a cosmetic control. Above the heating range, it will increase bubble emission rate, size variation, and upward speed. When a bubble intersects the height field, the system will apply a small impulse and remove or recycle the particle. Below the freezing range, a spatial ice mask will expand across the surface. That mask will increase damping and reduce displacement locally; at full coverage it will stop wave updates and constrain objects. This controller and its mappings from temperature to simulation coefficients form the main part of our custom integration. The renderer will use the same state to interpolate water and ice material parameters, keeping simulation and shading synchronized throughout the transition.

For rendering, the glass and water will use transparent materials with Fresnel-weighted reflection and refraction [5], while animated height-field normals will reveal small waves. Instanced geometry or particles will render many bubbles efficiently. Ice will use higher roughness, lower transmission, and subtle color or opacity variation so the frozen area remains distinct from the glass. We will evaluate the project through scenario-based tests covering cup motion, mouse ripples, floating and sinking objects, heating, boiling, partial freezing, and complete freezing. To demonstrate our contribution, we will also compare the integrated result with baselines in which temperature changes only the material or bubbles do not feed impulses back into the surface. Frame-time and stability tests at increasing grid and particle resolutions will determine settings that keep the complete scene responsive. We will keep a record of AI-assisted suggestions and identify which parts were accepted, modified, or rejected so that implementation ownership is clear in the final report.

References

[1] M. Kass and G. Miller, "Rapid, Stable Fluid Dynamics for Computer Graphics," *Proceedings of SIGGRAPH 1990*, pp. 49–57, 1990. doi: 10.1145/97879.97884.

[2] N. Chentanez and M. Müller, "Real-time Simulation of Large Bodies of Water with Small Scale Details," *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, pp. 197–206, 2010. doi: 10.2312/SCA/SCA10/197-206.

[3] W. T. Reeves, "Particle Systems—A Technique for Modeling a Class of Fuzzy Objects," *ACM Transactions on Graphics*, vol. 2, no. 2, pp. 91–108, 1983. doi: 10.1145/357318.357320.

[4] T. Kellomäki, "Rigid Body Interaction for Large-Scale Real-Time Water Simulation," *International Journal of Computer Games Technology*, vol. 2014, Article 580154, 2014. doi: 10.1155/2014/580154.

[5] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4th ed. Cambridge, MA: MIT Press, 2023.

[6] Three.js Contributors, "Water," *Three.js Documentation*. `https://threejs.org/docs/pages/Water.html`. Accessed September 28, 2026.

[7] Three.js Contributors, "Three.js Compute Water," *Three.js Examples*. `https://threejs.org/examples/webgpu_compute_water.html`. Accessed September 28, 2026.